CSC 718: Operating Systems & Parallel Programming

Assignment 2

Master-Worker Prime Counting Using MPI

Overview

In this assignment, you will build an MPI master-worker program that counts prime numbers in the range [2, N]. The emphasis is on process coordination, point-to-point communication, load distribution, correctness, and performance analysis on the Rushmore cluster.

The assignment has three progressive requirements:

- **R1:** Implement static master-worker scheduling and verify correctness.
- **R2:** Implement dynamic scheduling and compare it with the static version.
- **R3:** Run a scaling study on Rushmore and analyze performance.

Learning Goals

By completing this assignment, you will be able to:

- Use MPI processes, ranks, and communicators to coordinate distributed work.
- Implement point-to-point messaging patterns using MPI_Send and MPI_Recv.
- Design static and dynamic work-distribution strategies.
- Evaluate the trade-off between load balance and communication overhead.
- Measure execution time and analyze scaling on a distributed-memory cluster.

Problem Description

Count the number of prime numbers in the range [2, N]. Use a deterministic per-integer primality test. Trial division up to sqrt(k) is sufficient. Correctness, communication design, and performance reasoning matter more than low-level optimization.

The work is divided into contiguous segments. The command-line parameter CHUNK controls the number of integers in each segment and therefore the granularity of the distributed workload.

Required Command-Line Interface

Your executable must accept:

./primes_mw N CHUNK

- N: Upper bound of the range [2, N].
- CHUNK: Number of integers in each work unit.

Example build and run:

mpicc -o primes_mw primes_mw.c

mpirun -np 4 ./primes_mw 100000000 50000

Technical Constraints

- Use C or C++ with the MPI C API. Compile with mpicc or mpic++.
- Use point-to-point communication only in R1 and R2. Do not use MPI collectives such as MPI_Bcast, MPI_Reduce, MPI_Scatter, MPI_Gather, or MPI_Alltoall.

- Core MPI functionality should use MPI_Init, MPI_Comm_size, MPI_Comm_rank, MPI_Send, MPI_Recv, and MPI_Finalize. MPI_Wtime may be used for timing.
- Graded submissions must use C or C++. mpi4py may be explored independently but is not permitted for submission.
- Keep segment boundaries inside [2, N] and handle edge cases such as N < 2, small CHUNK values, and a final partial segment.

R1: Static Master-Worker Scheduling (40 pts)

Implement a static master-worker design using a predetermined round-robin mapping of work segments to workers.

Requirements

- Rank 0 is the master. Ranks 1 through P-1 are workers.
- The master partitions [2, N] into contiguous CHUNK-sized segments.
- Before execution begins, each segment is assigned to a worker using a fixed round-robin mapping.
- The master pushes the assigned segments to workers using MPI_Send. Workers do not request work in R1.
- Each worker receives its assigned segments, counts primes, and returns its partial count to the master using MPI_Send.
- The master receives all worker results using MPI_Recv and computes the final total.
- Use clear message tags such as WORK, RESULT, and STOP, or another consistent protocol.
- Terminate all workers cleanly after the assigned work has been completed.

Correctness Check

Run at least one small case, such as N = 200000. Rank 0 must also compute the prime count sequentially and verify that the distributed result matches the sequential result.

Small-Run Execution Evidence

For a small demonstration run, print brief segment-level logs that show how work is assigned and processed. For example:

master: send [200000..249999] to worker 5

worker 5: processed [200000..249999] -> 2543 primes

Detailed segment-level logging is for correctness and protocol evidence only. Disable it during timed performance experiments because frequent console output can distort execution time.

R2: Dynamic Scheduling (40 pts)

Upgrade the program to a dynamic master-worker design in which workers request new work when they become idle.

Requirements

- Workers use a pull model. A worker sends a REQUEST when it is ready for more work.
- The master responds with the next available segment or a STOP message when no work remains.
- After processing a segment, the worker returns a RESULT and requests additional work.
- Use a clear request-work-result-stop protocol with consistent message tags.
- The master must continue until all segments have been assigned, all results have been received, and all workers have been stopped.

- The program must terminate without deadlocks, lost results, or workers left waiting for messages.

Static vs. Dynamic Comparison

For one representative larger problem size, run both R1 and R2 using the same process count, CHUNK value, input range, and timing method. Compare their execution times and briefly explain the observed difference.

Do not assume that dynamic scheduling must be faster. Dynamic scheduling may improve load balance, but it also introduces additional communication. Your interpretation should consider both effects.

R3: Rushmore Scaling Study and Report (20 pts)

Evaluate the dynamic version from R2 on the Rushmore cluster. Use problem sizes large enough to produce measurable execution times.

Required Process Configurations

- 4 processes: 1 process per Rushmore VM.
- 8 processes: 2 processes per Rushmore VM.
- 16 processes: 4 processes per Rushmore VM.

Use the course Rushmore hostfile or scheduler configuration so that the required placement is maintained.

Problem Sizes and CHUNK

Use at least one small sanity run and two larger performance runs. Suggested values are:

- Small: N = 10,000,000.
- Medium: N = 50,000,000.
- Large: N = 100,000,000.

Choose a reasonable CHUNK value, such as 20,000 or 50,000. Use the same CHUNK for a given comparison and briefly explain your choice. If the suggested N values do not produce useful timing on the current Rushmore environment, you may increase them while documenting the values used.

Timing Requirement

Use MPI_Wtime to measure only the parallel work-distribution and computation phase.

- Start the timer after MPI initialization and setup, immediately before the first work dispatch or request-work exchange.
- Stop the timer after the master has received all results and before MPI_Finalize.
- Keep detailed segment-level logging disabled during timed runs.
- Use the same timing region for every configuration.

Proof of Placement

At startup, each process must print its rank, total process count, and hostname using gethostname(). Include execution evidence that confirms the required process placement for the 4-, 8-, and 16-process configurations.

Performance Measures

Use the 4-process configuration as the relative baseline for each problem size.

- Relative speedup: $S_4(P) = T(4) / T(P)$.
- Relative efficiency: $E_4(P) = S_4(P) / (P/4)$.

For P = 4, both relative speedup and relative efficiency are 1.0. These metrics show how performance changes as resources increase from the 4-process baseline.

Required Report Content

Submit a PDF report containing:

- A brief description of the master-worker design and message protocol for R1 and R2.
- Correctness evidence showing that the distributed result matches the sequential result for a small case.
- The controlled static-versus-dynamic comparison from R2.
- A timing table for P in {4, 8, 16} for each selected problem size.
- Relative speedup and relative efficiency calculations.
- Execution evidence showing commands, host placement, final prime counts, and timing summaries.
- A concise interpretation of scaling, load balance, communication overhead, CHUNK size, and intra-node contention when moving from 1 to 4 processes per VM.

For timed runs, summary output should remain brief. A final summary should include N, P, CHUNK, total prime count, and elapsed time.

Submission Requirements

Upload a single ZIP file to D2L containing the following.

Source Code

- C or C++ source files for R1 and R2, either as separate programs or one codebase with a clear mode or flag.
- Any additional header or helper files required to build the program.

README

Include:

- Exact build instructions.
- Exact run instructions, including Rushmore hostfile or scheduler usage.
- Meaning of N and CHUNK.
- Description of the primality test.
- Description of the timing method.
- Any assumptions, limitations, or implementation notes.

Report

Include the PDF report described in R3 with correctness evidence, the static-versus-dynamic comparison, timing results, relative speedup and efficiency, placement evidence, and performance discussion.

AI Use

Limited use of generative AI tools is permitted for activities such as clarifying syntax, exploring approaches, debugging, or other technical support.

AI tools must not be used to generate substantial portions of the submitted code, performance analysis, report text, results, screenshots, logs, or other execution evidence.

If you use an AI tool, include a brief AI Use Statement in your README. Identify the tool used and briefly describe how it assisted you. If no AI tools were used, state this explicitly.

You are responsible for understanding, testing, and verifying all submitted work.

Grading Rubric (100 pts)

R1: Static Master-Worker Scheduling (40 pts)

- Correct predetermined round-robin work distribution using MPI_Send and MPI_Recv only: 15 pts
- Workers compute partial counts and the master produces the correct total: 15 pts
- Small-N correctness check against a sequential count: 5 pts
- Code organization, readability, and appropriate comments: 5 pts

R2: Dynamic Scheduling (40 pts)

- Correct request-work-result-stop protocol using point-to-point communication: 20 pts
- Controlled static-versus-dynamic comparison with technically sound interpretation: 10 pts
- Clean termination with no deadlocks, lost results, or waiting workers: 5 pts
- Code organization, readability, and appropriate comments: 5 pts

R3: Rushmore Scaling Study and Report (20 pts)

- Required P = 4, 8, 16 Rushmore runs with correct process placement and execution evidence: 8 pts
- Timing table with correct relative speedup and relative efficiency calculations: 8 pts
- Concise analysis of scaling, CHUNK size, communication, load balance, and intra-node contention: 4 pts